الجامعة الدولية للرباط
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⴳⵔⴰⵖⵍⴰⵏⵜ ⵏ ⵕⵕⴱⴰⵟ
Université Internationale de Rabat

Web Avancé et Développement mobile

PROF. RAHHAL Ibrahim
EMAIL: BRAHIM.RAHHAL@UIR.AC.MA

Année Universitaire: 2023-2024

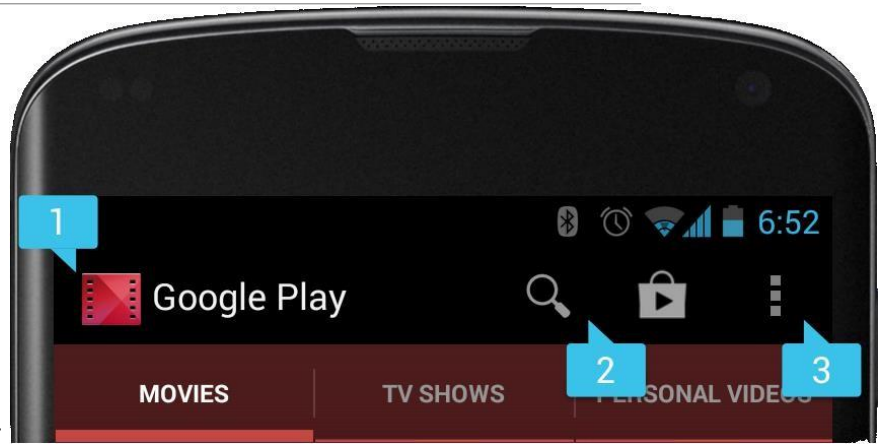
Développement Mobile sous Android

Ergonomie d’une application Android

Barre d’action et menus

La barre d’action contient

1. l’icône d’application
2. des items de menu
3. un bouton pour avoir les autres menus.



Le principe général d’un menu : une liste d'items qui apparait soit quand on appuie sur le bouton menu, soit sur la barre d'action. Certains de ces items sont présents en permanence dans la barre d'action. La sélection d'un item déclenche une callback.

Pour Réaliser un menu

- Spécifier le fichier res/menu/nom_du_menu.xml,
- Choisir le thèmes pour afficher les menus,
- Implémenter les deux callbacks pour gérer les menus : création et activation.

Barre d’action et menus : Spécification

Créer la Spécification d’un menu dans le dossier Ressource (res/menu/nom_du_menu.xml):

```
<menu xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/menu_creer"
        android:icon="@drawable/ic_menu_creer"
        android:title="Créer"
        app:showAsAction="ifRoom" />
    <item android:title="Modifier"

        />
</menu>
```

L’attribut showAsAction vaut

- « always », withText
- « ifRoom », CollapseActionView
- « never »

selon la visibilité qu’on souhaite dans la barre d’action.

Android Propose gratuitement un grand jeu d'icones pour les menus, dans les deux styles MaterialDesign : HoloDark et HoloLight.

Barre d'action et menus : Spécification

Écouteurs pour les menus : Il faut programmer deux méthodes.

- onCreateOptionsMenu(Menu menu) : permet d'afficher le menu (Cette méthode rajoute les items du menu défini dans le XML.)

```
@Override
public boolean onCreateOptionsMenu(Menu menu)
{
    // ajouter mes items de menu
    getMenuInflater().inflate(R.menu.menu_general, menu);
    // ajouter les items du système s'il y en a
    return super.onCreateOptionsMenu(menu);
}
```

Un MenuInflater est un lecteur/traducteur de fichier XML en vues : sa méthode inflate crée les vues.

- onOptionsItemSelected(MenuItem item): réagit quand l'utilisateur sélectionne un item.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.menu_creer:

            return true;
        case R.id.menu_modifier:

        default: return super.onOptionsItemSelected(item);
    }
}
```

Barre d’action et menus : Menus en cascade

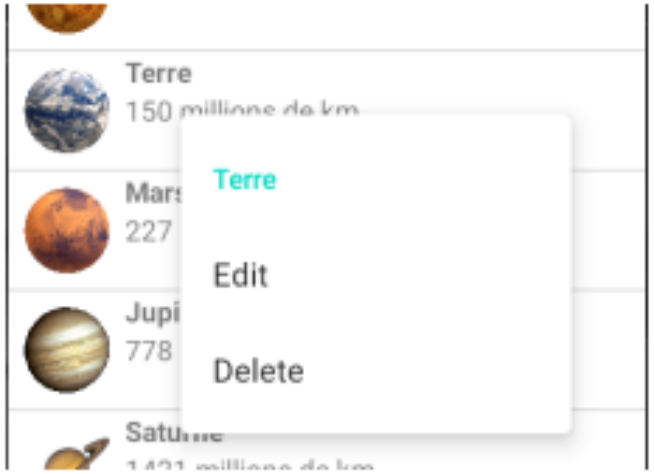
On peut spécifier Plusieurs niveaux quand la barre d’action est petite:

```
<menu xmlns:android2="http://schemas.android.com/apk/res/android"
    xmlns:android="..." >
    <item android:id="@+id/menu_item1"
        android2:title="item1" />
    <item android:id="@+id/menu_item2"
        android2:title="item2" />
    <item android:id="@+id/menu_more"
        android2:title="item"
        android:icon="@drawable/ic_action_overflow"
        android:showAsAction="always"
        android:title="@string/menu_more">
        <menu>
            <item android:id="@+id/menu_item3"
                android2:title="item3"/>
            <item android:id="@+id/menu_item4"
                android2:title="item4"/>
        </menu>
    </item>
</menu>
```

Barre d’action et menus : Menus contextuels

Menus contextuels apparaissent lors un clic long sur un élément de liste. Le principe est le même que pour les menus normaux :

- Attribuer un écouteur à l’événement onCreateContextMenu. événement correspond à un clic long : fait apparaître le menu.
- Définir la callback de l’écouteur : elle expanse un layout de menu.
- Définir la callback des items du menu.



Barre d'action et menus : Menus contextuels

Associer un menu contextuel à une vue : dans la méthode onCreate d'une activité :

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    ListView lv = findViewById(android.R.id.list);
    registerForContextMenu(lv); // ou utiliser lv.setOnCreateContextMenuListener(this);
```

Callback d'affichage du menu : Un clic long sur un élément de la liste déclenche cette méthode :

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenu.ContextMenuInfo menuInfo)
{
    super.onCreateContextMenu(menu, v, menuInfo);
    getMenuInflater().inflate(R.menu.menu_general, menu);
}
```

Barre d’action et menus : Menus contextuels

Callback des items du menu : si l’utilisateur choisit un item du menu :

```
public boolean onContextItemSelected(MenuItem item) {
    AdapterView.AdapterContextMenuInfo info = (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();
    switch (item.getItemId()) {
        case R.id.menu_item1:
            onMenuEditer(info.id); // identifiant de l'élément
            return true;
        case R.id.menu_item2:
            onMenumodifier(info.id);
            return true;
    }
    return false;
}
```

Annonces et dialogues : toasts

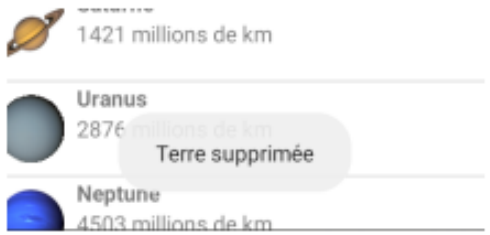
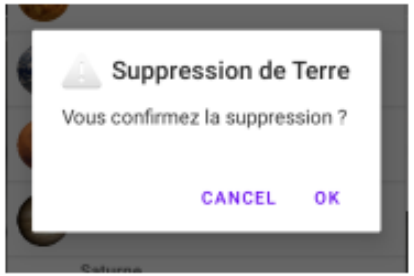


Figure 30: Toast

Un « toast » est un message apparaissant en bas d'écran pendant un instant, par exemple pour confirmer la réalisation d'une action. Un toast n'affiche aucun bouton et n'est pas actif.:



On peut l'afficher avec une ressource chaîne : LENGTH_... Specifie la durée d'affichage :

```
Toast.makeText(getContext(), R.string.item_supprimee, Toast.LENGTH_SHORT).show();
```

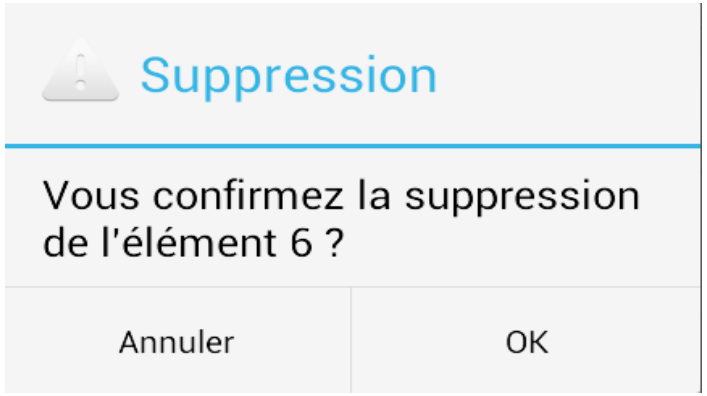
Pour personnaliser une annonce, on peut définir un layout dans res/layout/toast_perso.xml. La racine de ce layout doit avoir un identifiant, qui est mentionné dans la création :

```
// expanser le layout du toast
LayoutInflater inflater = getLayoutInflater();
View layout = inflater.inflate(R.layout.toast_perso,
        (ViewGroup) findViewById(R.id.toast_perso_id));

// créer le toast et l'afficher
Toast toast = new Toast(getApplicationContext());
toast.setDuration(Toast.LENGTH_LONG);
toast.setView(layout);
toast.show();
```

Annonces et dialogues : Dialogues

Un dialogue est une petite fenêtre qui apparaît au dessus d'un écran pour afficher ou demander quelque chose d'urgent à l'utilisateur, par exemple une confirmation:



Il existe plusieurs sortes de dialogues :

- Dialogues d'alerte
- Dialogues généraux

Annonces et dialogues : Dialogue d'alerte

Un dialogue d'alerte AlertDialog affiche un texte et un à trois boutons au choix : ok, annuler, oui, non, aide. . .

Un dialogue d'alerte est construit à l'aide d'une classe nommée AlertDialog.Builder. Le principe est de créer un builder et c'est lui qui crée le dialogue :

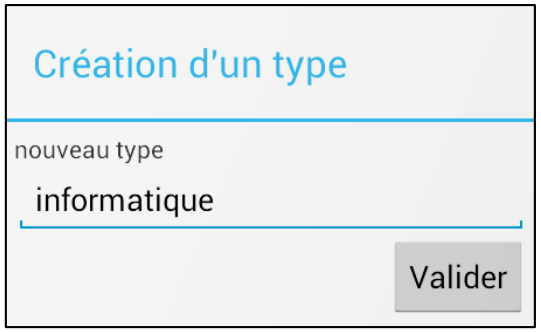
```
AlertDialog.Builder builder = new AlertDialog.Builder( context: this);
builder.setTitle("Suppression");
builder.setIcon(android.R.drawable.ic_dialog_alert);
builder.setMessage("Vous confirmez la suppression ?");
```

Ensuite, on rajoute les boutons et leurs écouteurs:

```
// rajouter un bouton "oui" qui supprime vraiment
builder.setPositiveButton(android.R.string.yes,
// écouteur écrit sous la forme d'une lambda
        (dialog, idbtn) -> {
            SupprimerElement(idElement);
});
// rajouter un bouton "non" qui ne fait rien
builder.setNegativeButton(android.R.string.no,  listener: null);
// affichage du dialogue
builder.show();
```

Annonces et dialogues : Dialogues personnalisés

Lorsqu'il faut demander une information plus complexe à l'utilisateur, mais sans que ça nécessite une activité à part entière, il faut faire appel à un dialogue personnalisé.



Création d'un dialogue: définir le layout du dialogue incluant tous les textes, et un bouton pour valider et on peut fermer le dialogue avec le bouton back.

```
<LinearLayout xmlns:android="..."
    android:layout_height=""
    android:layout_width=""
    xmlns:android="http://schemas.android.com/apk/res/android" >
        <TextView android:id="@+id/dlg_titre" .../>
        <EditText android:id="@+id/dlg_libelle" .../>
    <Button android:id="@+id/dlg_btn_valider" ... />
    </LinearLayout>
```

Annonces et dialogues : Dialogues personnalisés

Affichage du dialogue : cela ressemble à ce qu'on fait dans onCreate d'une activité : setView avec le layout et des setOnClickListener pour attribuer une action aux boutons.

```
// créer le dialogue (final car utilisé dans les écouteurs)
final Dialog dialog = new Dialog( context: this);
dialog.setView(R.layout.item);
dialog.setTitle("Création d'un type");
// bouton valider
Button btnValider = dialog.findViewById(R.id.dlg_btn_valider);
btnValider.setOnClickListener(v -> {
// récupérer et traiter les infos

// ne surtout pas oublier de fermer le dialogue
    dialog.dismiss();
    }
});
// afficher le dialogue
dialog.show();
```

Fragments et activités

Un fragment est un sous-ensemble d'une interface d'application, par exemple :

- liste d'items
- affichage des infos d'un item
- formulaire d'édition d'un item

Un fragment est une sorte de mini-activi

informations d'un élément, etc.

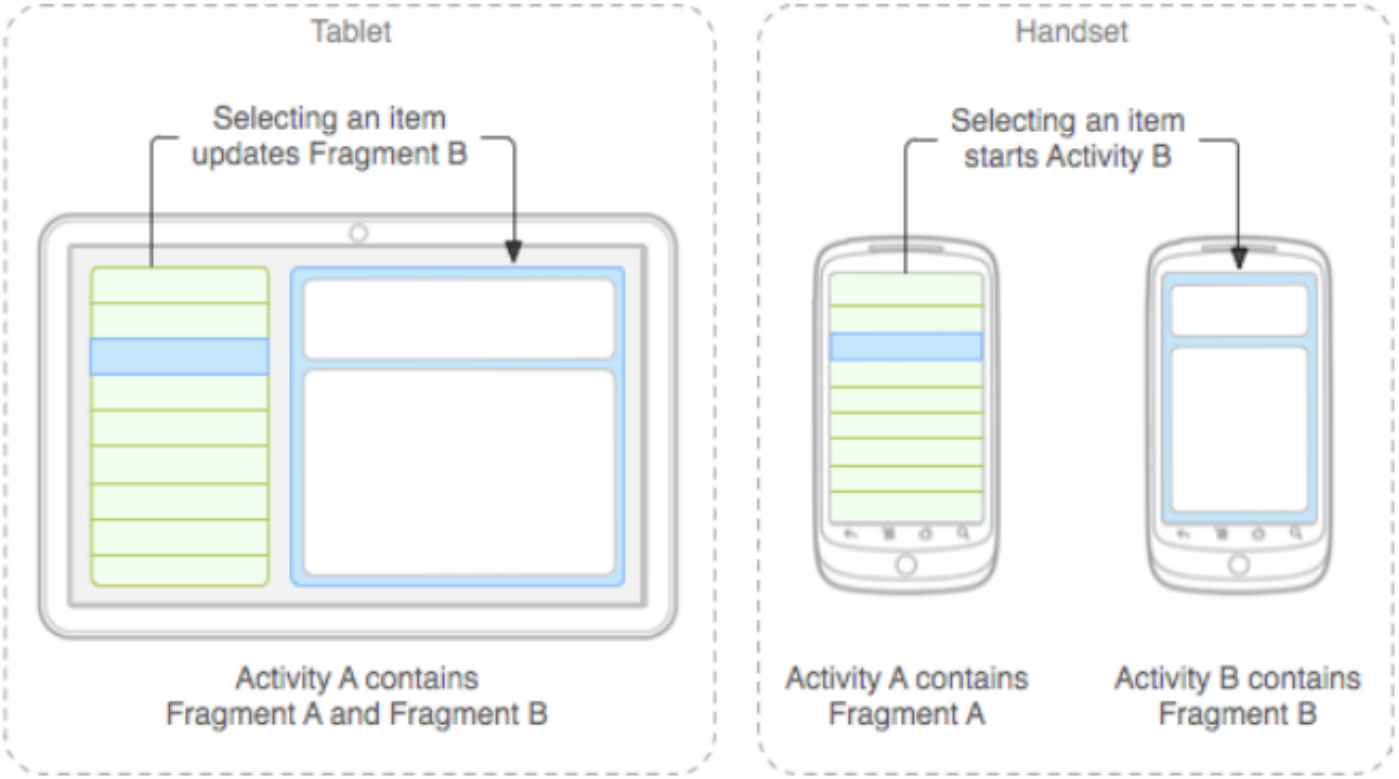
Une activité peut être composée d'un ou

géométrie du smartphone.

Une interface devient plus souple avec

liste et

les détails, ou séparer les deux.



Fragments et activités : Structure d'un fragment

Un fragment est une activité très simplifiée qui contient au moins un constructeur vide et onCreateView surchargée :

```
public class InfosFragment extends Fragment {
  public InfosFragment() {}  // obligatoire

  @Override
  public View onCreateView(LayoutInflater inflater,
            ViewGroup container, Bundle savedInstanceState) {
    ui = PlaneteInfosBinding.inflate(inflater, container, false);

    // écouteurs, adaptateur...

    return ui.getRoot();
  }
}
```

Fragments et activités : Intégrer un fragment dans une activité

De lui-même, un fragment n'est pas capable de s'afficher. Il ne peut apparaître que dans le cadre d'une activité, comme une sorte de vue interne. On peut le faire de deux manières :

- statiquement : les fragments à afficher sont prévus dans le layout de l'activité. C'est le plus simple.
- dynamiquement : les fragments sont ajoutés, enlevés ou remplacés en cours de route selon les besoins (on ne verra pas)

Dans ce cas, c'est le layout de l'activité qui inclut les fragments, p. ex. res/layout/activity_main.xml. Ils ne peuvent pas être modifiés ultérieurement.

```
<LinearLayout android:orientation="horizontal" ... >
    <fragment   android:id="@+id/frag_liste"
        android:name="fr.iutlan.fragments.ListeFragment"
        ... />
    <fragment   android:id="@+id/frag_infos"
        android:name="fr.iutlan.fragments.InfosFragment"
```

Chaque fragment doit avoir un identifiant et un nom de classe complet avec tout le package. Ne pas oublier les attributs des tailles et éventuellement poids.

Fragments et activités : Intégrer un fragment dans une activité

- Changer la disposition selon la géométrie

Pour cela, il suffit de définir deux layouts (des « variantes ») :

- res/layout-port/activity_main.xml en portrait
- res/layout-land/activity_main.xml en paysage

Lorsque la tablette est verticale, le layout de layout-port est affiché et lorsqu'elle est horizontale, c'est celui de layout-land.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    ...
    FragmentManager manager = getSupportFragmentManager();
    InfosFragment frag_infos = (InfosFragment)
                manager.findFragmentById(R.id.frag_infos);
    if (frag_infos != null) {
        // le fragment des informations est présent
        ...
```